Assignment 05

Question 01

In [7]:
```
import numpy as np

# Sample data from assignment
X = np.array([6.1101, 5.5277, 8.5186])
y = np.array([17.592, 9.1302, 13.662])
m = len(y)

# Add intercept term to X
Xb = np.c_[np.ones(m), X]

# Initialize fitting parameters
theta = np.zeros(2)

# Gradient descent settings
alpha = 0.01
iterations = 1500

def compute_cost(X, y, theta):
    errors = X @ theta - y
    return (1 / (2 * m)) * np.dot(errors, errors)

def gradient_descent(X, y, theta, alpha, iterations):
    for _ in range(iterations):
        gradient = (1 / m) * X.T @ (X @ theta - y)
        theta = theta - alpha * gradient
    return theta

theta = gradient_descent(Xb, y, theta, alpha, iterations)
print("Learned theta:", theta)
```

```
Learned theta: [4.02750103 1.37581788]
```

Question 02

In [8]:
```
import numpy as np

# Sample data
X = np.array([[2104, 5], [1600, 3], [2400, 4]])
y = np.array([399900, 329900, 369000])
m = len(y)

# Feature normalization
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_norm = (X - X_mean) / X_std

# Add intercept term
Xb = np.c_[np.ones(m), X_norm]
```

```
# Initialize fitting parameters
theta = np.zeros(Xb.shape[1])

# Gradient descent settings
alpha = 0.01
iterations = 400

def gradient_descent(X, y, theta, alpha, iterations):
    for _ in range(iterations):
        gradient = (1 / m) * X.T @ (X @ theta - y)
        theta = theta - alpha * gradient
    return theta

theta = gradient_descent(Xb, y, theta, alpha, iterations)
print("Learned theta:", theta)
```

```
Learned theta: [359691.97735379   5161.17119473  24305.29407282]
```

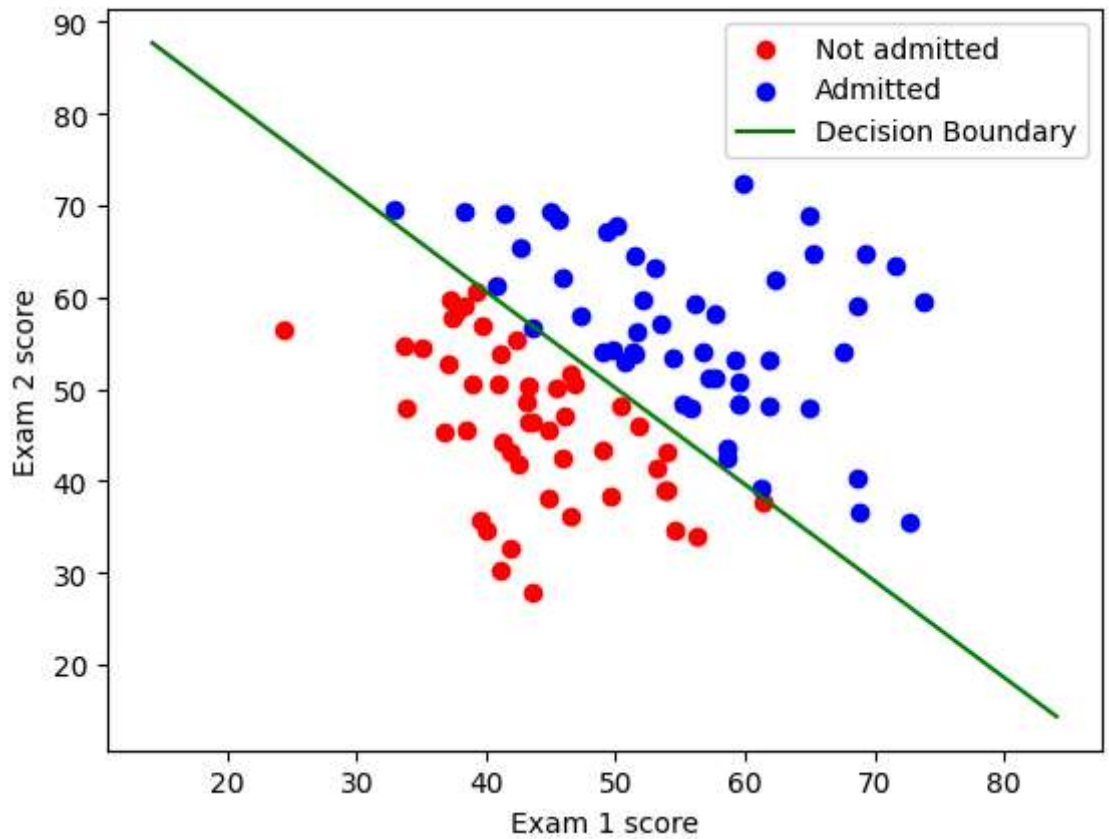
Question 03

In [9]:

```
import numpy as np
import matplotlib.pyplot as plt

# Generate linearly separable exam scores and admission results
np.random.seed(0)
X = np.random.randn(100, 2) * 10 + 50  # Exam 1 and Exam 2 scores
y = (X[:, 0] + X[:, 1] > 100).astype(int)  # Admission if sum > 100

# Feature scaling (standardization)
X_mean = np.mean(X, axis=0)
X_std = np.std(X, axis=0)
X_norm = (X - X_mean) / X_std

# Add intercept term
m = len(y)
Xb = np.c_[np.ones(m), X_norm]

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_cost(X, y, theta):
    h = sigmoid(X @ theta)
    epsilon = 1e-5
    cost = -(1 / m) * np.sum(y * np.log(h + epsilon) + (1 - y) * np.log(1 - h + epsil
    return cost

def gradient_descent(X, y, theta, alpha, iterations):
    for i in range(iterations):
        h = sigmoid(X @ theta)
        gradient = (1 / m) * X.T @ (h - y)
        theta = theta - alpha * gradient
        if i % 500 == 0:
            print(f"Iteration {i}: Cost {compute_cost(X, y, theta)}")
    return theta
```

```python
theta = np.zeros(Xb.shape[1])
alpha = 0.05
iterations = 5000
theta = gradient_descent(Xb, y, theta, alpha, iterations)
print("Learned theta:", theta)

# Plot data points
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], c='red', label='Not admitted')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], c='blue', label='Admitted')

# Plot decision boundary
x_vals = np.linspace(np.min(X_norm[:, 0]) - 1, np.max(X_norm[:, 0]) + 1, 100)
y_vals = -(theta[0] + theta[1] * x_vals) / theta[2]

# Convert normalized values back to original for plot axes
x_vals_orig = x_vals * X_std[0] + X_mean[0]
y_vals_orig = y_vals * X_std[1] + X_mean[1]

plt.plot(x_vals_orig, y_vals_orig, 'g-', label='Decision Boundary')
plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.legend()
plt.show()

# Evaluate accuracy
preds = sigmoid(Xb @ theta) >= 0.5
accuracy = np.mean(preds == y) * 100
print(f'Accuracy on training set: {accuracy:.2f}%')
```

```
Iteration 0: Cost 0.6853629723341312
Iteration 500: Cost 0.18999529034037163
Iteration 1000: Cost 0.1417533537524454
Iteration 1500: Cost 0.11979766335129961
Iteration 2000: Cost 0.10650796106391458
Iteration 2500: Cost 0.09731945782493698
Iteration 3000: Cost 0.09045365663908875
Iteration 3500: Cost 0.08505546746520208
Iteration 4000: Cost 0.08065594075069264
Iteration 4500: Cost 0.07697340605605109
Learned theta: [0.66021139 5.50645242 5.20506843]
```




```
Accuracy on training set: 99.00%
```

Question 04

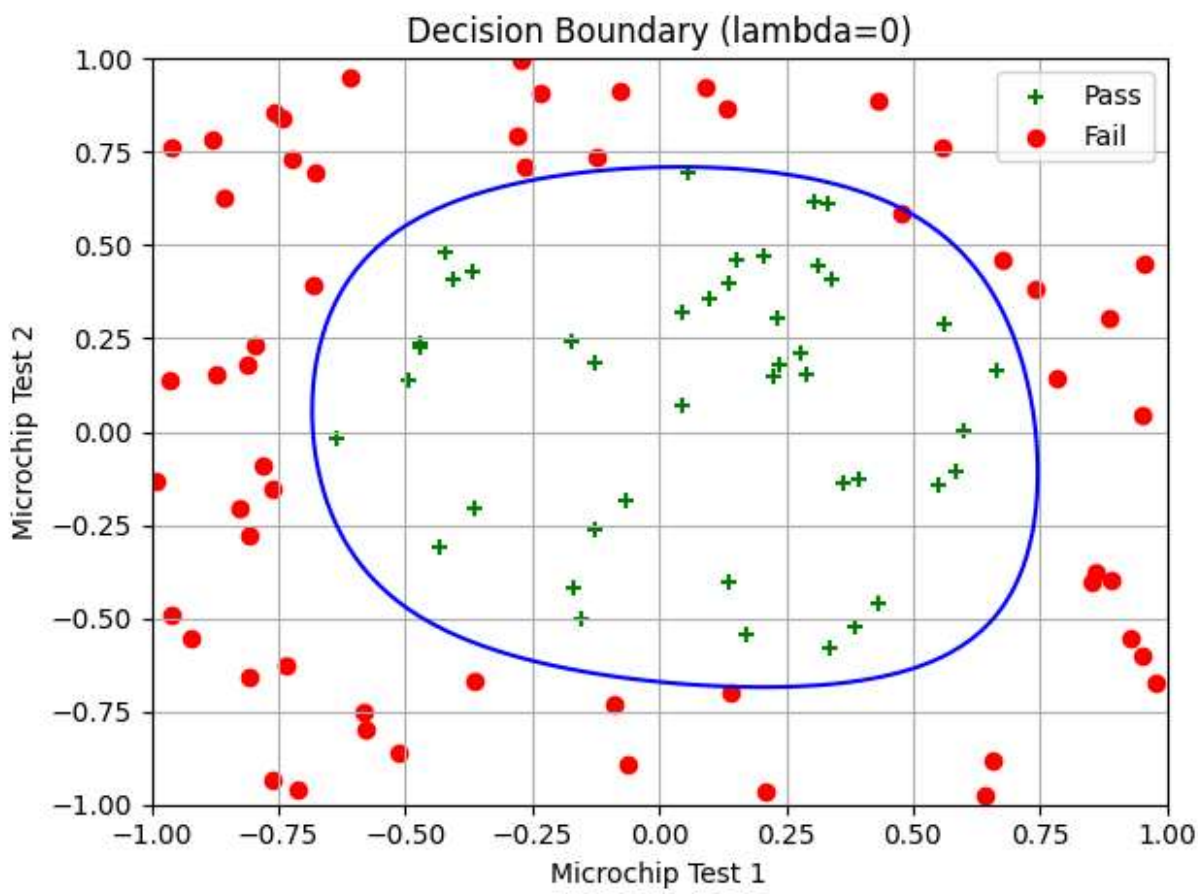
In [10]:

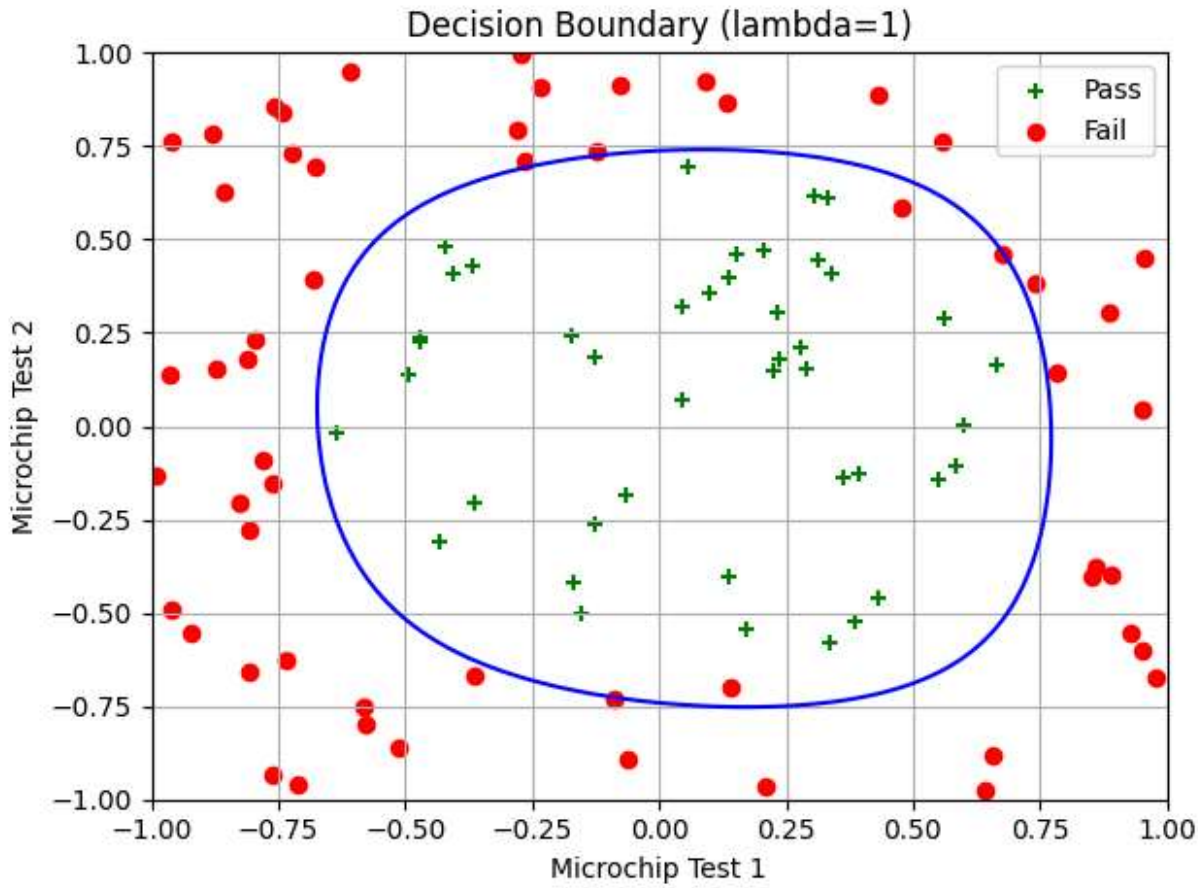
```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate Dataset
np.random.seed(0)
test1 = np.random.uniform(-1, 1, 100)
test2 = np.random.uniform(-1, 1, 100)

# Circular decision boundary logic (non-linear)
labels = ((test1**2 + test2**2) < 0.5).astype(int)

X = np.column_stack((test1, test2))
y = labels

# 2. Feature Mapping up to 6th degree
def map_features(x1, x2, degree=6):
    m = x1.shape[0]
    output = np.ones((m, 1))
    for i in range(1, degree+1):
        for j in range(i+1):
            term = (x1**(i-j) * x2**j).reshape(m, 1)
            output = np.hstack((output, term))
    return output

# 3. Sigmoid
def sigmoid(z):
```

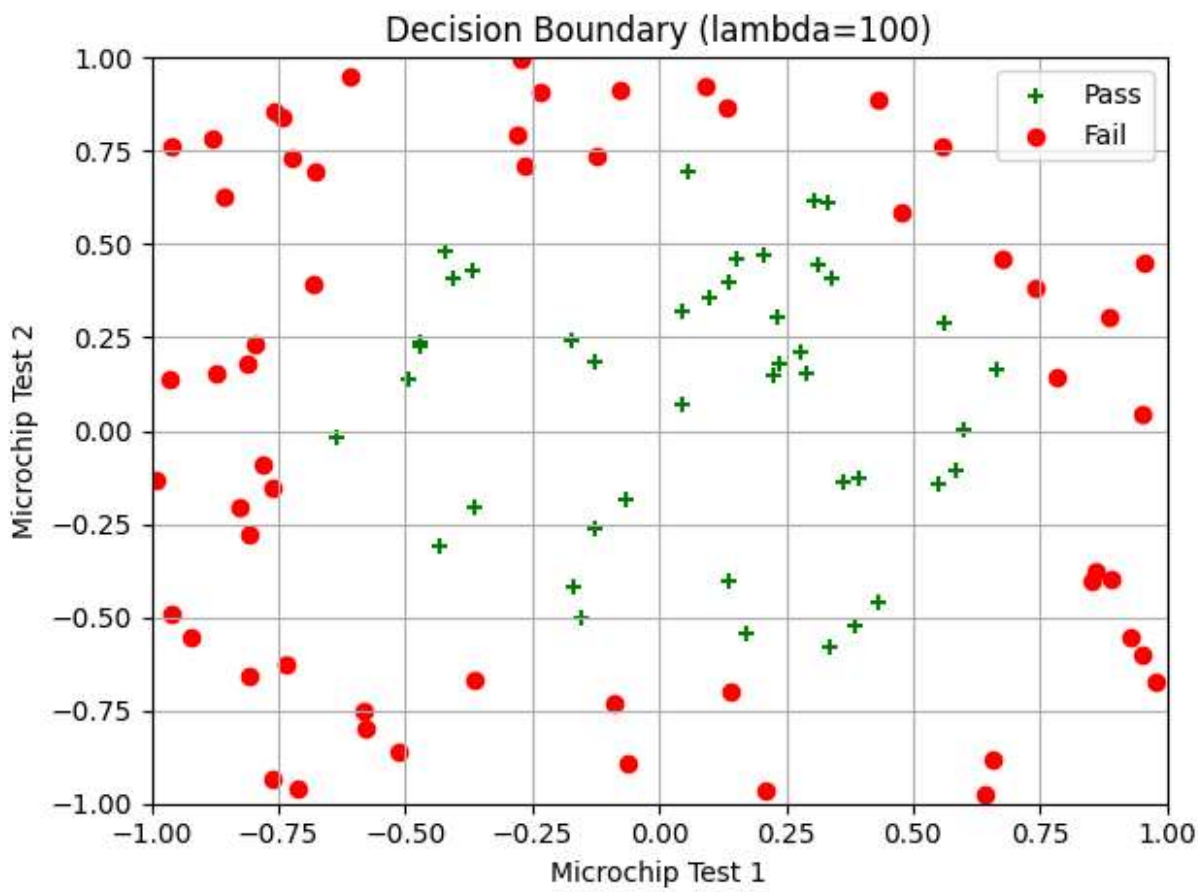
```
    return 1 / (1 + np.exp(-z))

# 4. Regularized Cost Function
def compute_cost_reg(X, y, theta, lambda_):
    m = len(y)
    h = sigmoid(X @ theta)
    reg_term = (lambda_ / (2 * m)) * np.sum(theta[1:] ** 2)
    cost = (-y @ np.log(h) - (1 - y) @ np.log(1 - h)) / m + reg_term
    return cost

# 5. Regularized Gradient Descent
def gradient_descent_reg(X, y, theta, alpha, num_iters, lambda_):
    m = len(y)
    cost_history = []

    for _ in range(num_iters):
        h = sigmoid(X @ theta)
        grad = (X.T @ (h-y)) / m
        grad[1:] += (lambda_ / m) * theta[1:]
        theta -= alpha * grad
        cost_history.append(compute_cost_reg(X, y, theta, lambda_))

    return theta, cost_history

# 6. Plot Decision Boundary
def plot_decision_boundary(X_raw, y, theta, lambda_):
    u = np.linspace(-1, 1, 100)
    v = np.linspace(-1, 1, 100)
    z = np.zeros((len(u), len(v)))

    for i in range(len(u)):
        for j in range(len(v)):
            mapped = map_features(np.array([u[i]]), np.array([v[j]]))
            z[i, j] = mapped @ theta

    plt.contour(u, v, z.T, levels=[0], colors='b')
    pos = y == 1
    neg = y == 0
    plt.scatter(X_raw[pos][:, 0], X_raw[pos][:, 1], c='g', marker='+', label='Pass')
    plt.scatter(X_raw[neg][:, 0], X_raw[neg][:, 1], c='r', marker='o', label='Fail')
    plt.title(f"Decision Boundary (lambda={lambda_})")
    plt.legend()
    plt.xlabel('Microchip Test 1')
    plt.ylabel('Microchip Test 2')
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# 7. Training for different λ values
X_mapped = map_features(X[:, 0], X[:, 1])
theta_init = np.zeros(X_mapped.shape[1])
lambda_values = [0, 1, 100]

for lambda_ in lambda_values:
    theta_opt, _ = gradient_descent_reg(X_mapped, y, theta_init.copy(), alpha=1, num_
    plot_decision_boundary(X, y, theta_opt, lambda_)
```

```
# 8. Effect of underfitting vs overfitting
print("""
Discussion:
- λ = 0: No regularization. The model overfits and decision boundary is very complex.
- λ = 1: Balanced regularization. Good generalization with smooth boundary.
- λ = 100: Too much regularization. The model underfits and cannot separate classes w
""")
```

```
C:\Users\Sabbir Ahmed\AppData\Local\Temp\ipykernel_21480\1379348335.py:60: Deprecation
Warning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will erro
r in future. Ensure you extract a single element from your array before performing thi
s operation. (Deprecated NumPy 1.25.)
  z[i, j] = mapped @ theta
```




Decision Boundary (lambda=1)
Pass
Fail
Microchip Test 2
Microchip Test 1
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00


Decision Boundary (lambda=100)
Pass
Fail
Microchip Test 2
Microchip Test 1
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

```
Discussion:
- λ = 0: No regularization. The model overfits and decision boundary is very complex.
- λ = 1: Balanced regularization. Good generalization with smooth boundary.
- λ = 100: Too much regularization. The model underfits and cannot separate classes we
ll.
```

Question 05

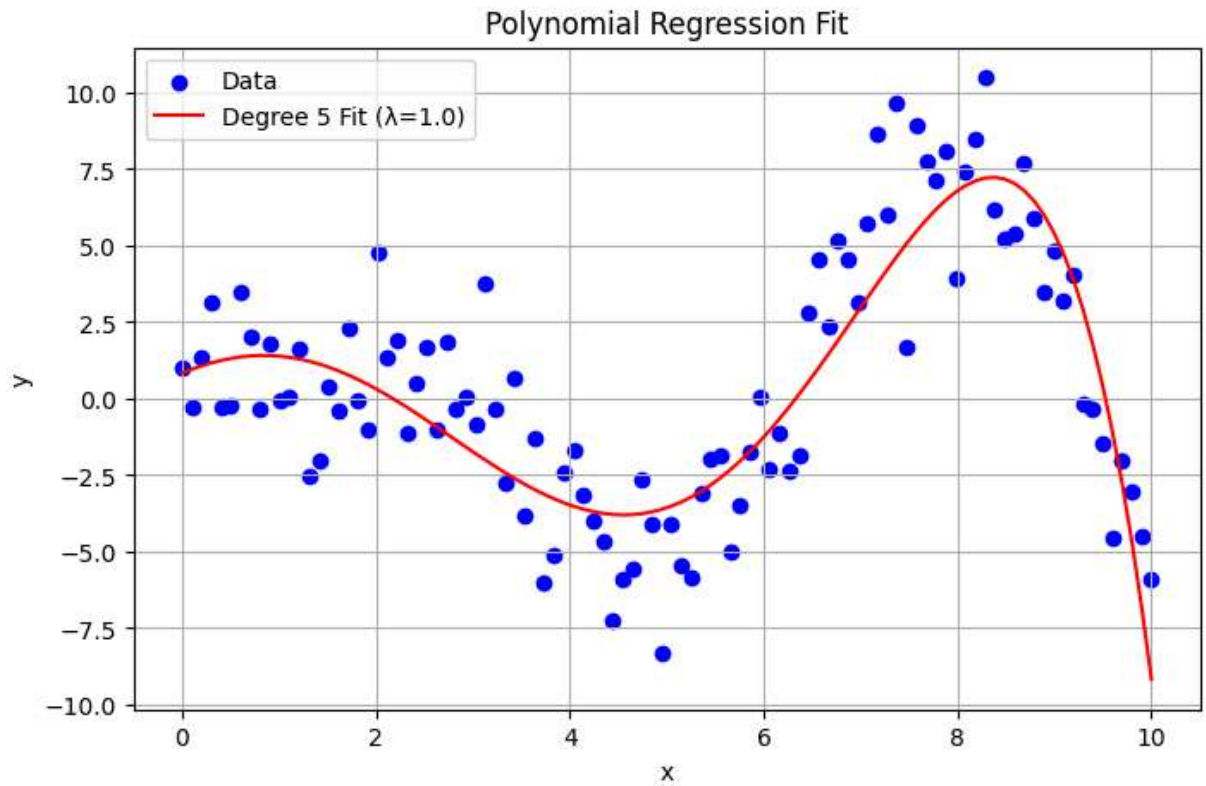
In [11]:

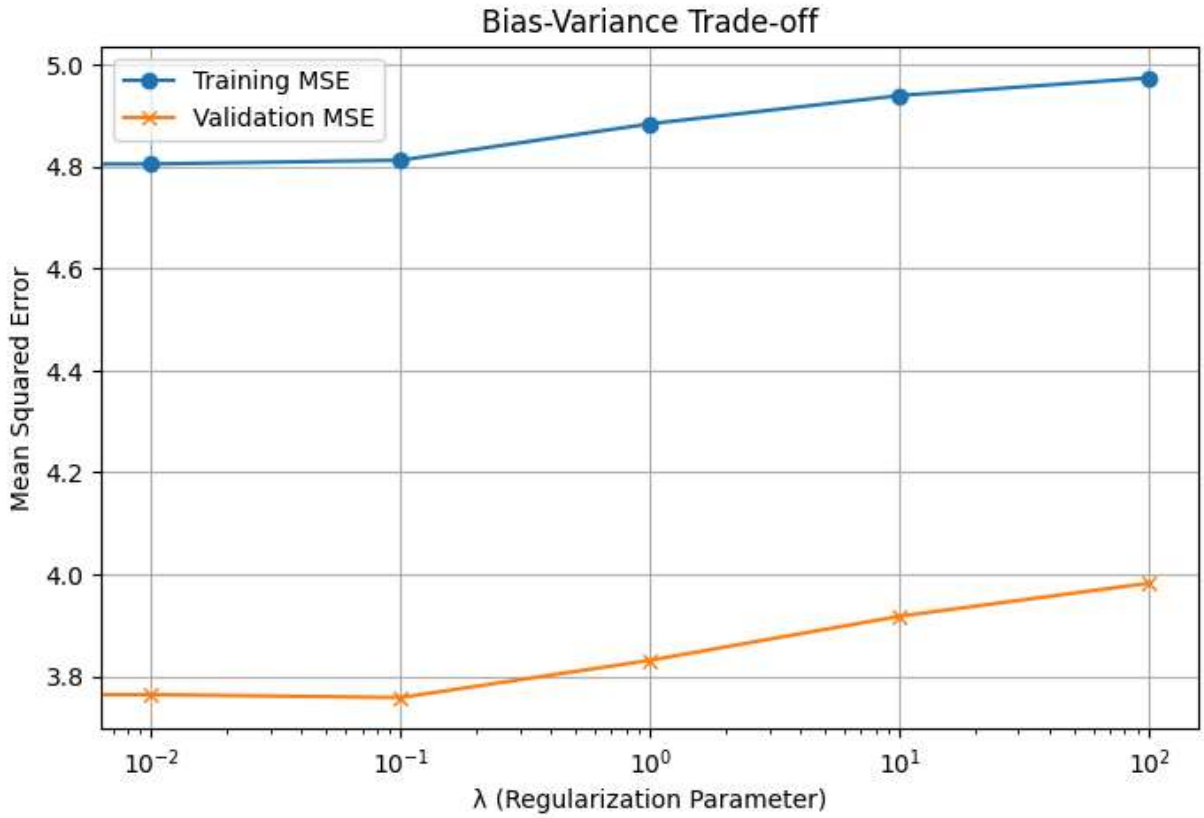
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Step 1: Generate synthetic non-linear dataset
np.random.seed(42)
X = np.linspace(0, 10, 100).reshape(-1, 1)
y = X.ravel() * np.sin(X.ravel()) + np.random.normal(0, 2, X.shape[0])

# Step 2: Polynomial feature transformation
poly_degree = 5
poly = PolynomialFeatures(degree=poly_degree)
X_poly = poly.fit_transform(X)

# Step 3: Split data into train and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_poly, y, test_size=0.3, random_st

# Step 4: Train model with Ridge regularization
lambda_val = 1.0
model = Ridge(alpha=lambda_val)
model.fit(X_train, y_train)

# Step 5: Predict for plotting
X_plot = np.linspace(0, 10, 100).reshape(-1, 1)
X_plot_poly = poly.transform(X_plot)
y_plot = model.predict(X_plot_poly)

# Print learned parameters
print("Learned Parameters (θ):", model.coef_)

# Step 6: Plot polynomial fit
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color='blue', label='Data')
plt.plot(X_plot, y_plot, color='red', label=f'Degree {poly_degree} Fit (λ={lambda_val
plt.xlabel('x')
plt.ylabel('y')
plt.title('Polynomial Regression Fit')
plt.legend()
plt.grid(True)
plt.show()

# Step 7: Plot learning curves
def plot_learning_curve(model, X_train, y_train, X_val, y_val, train_sizes):
```

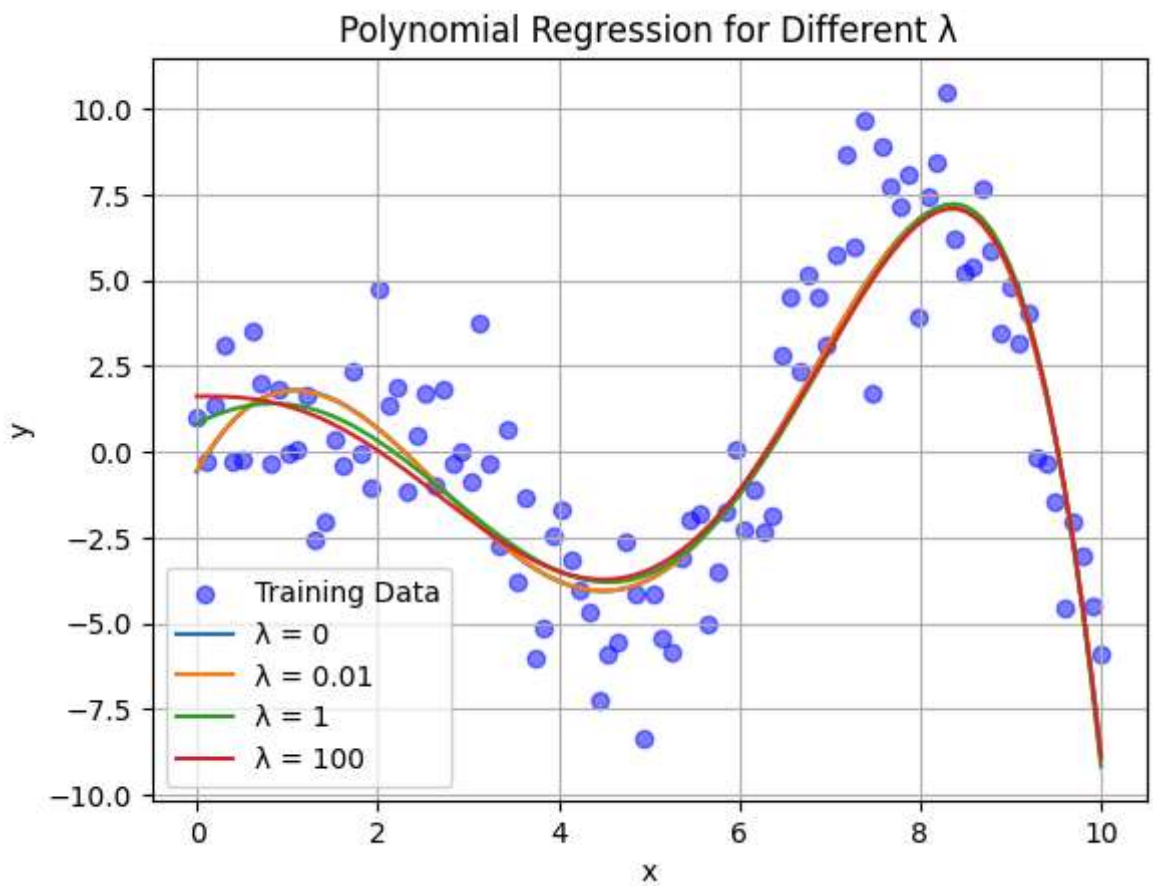
```
    train_errors = []
    val_errors = []

    for m in train_sizes:
        model.fit(X_train[:m], y_train[:m])
        train_pred = model.predict(X_train[:m])
        val_pred = model.predict(X_val)

        train_errors.append(mean_squared_error(y_train[:m], train_pred))
        val_errors.append(mean_squared_error(y_val, val_pred))

    plt.figure(figsize=(8, 5))
    plt.plot(train_sizes, train_errors, label='Training Error')
    plt.plot(train_sizes, val_errors, label='Validation Error')
    plt.xlabel('Training Set Size')
    plt.ylabel('Mean Squared Error')
    plt.title('Learning Curves')
    plt.legend()
    plt.grid(True)
    plt.show()

train_sizes = np.linspace(10, len(X_train), 10, dtype=int)
plot_learning_curve(Ridge(alpha=lambda_val), X_train, y_train, X_val, y_val, train_si

# Step 8: Analyze bias-variance trade-off by varying λ
lambdas = [0, 0.01, 0.1, 1, 10, 100]
train_mse = []
val_mse = []

for l in lambdas:
    model = Ridge(alpha=l)
    model.fit(X_train, y_train)
    train_mse.append(mean_squared_error(y_train, model.predict(X_train)))
    val_mse.append(mean_squared_error(y_val, model.predict(X_val)))

plt.figure(figsize=(8, 5))
plt.plot(lambdas, train_mse, marker='o', label='Training MSE')
plt.plot(lambdas, val_mse, marker='x', label='Validation MSE')
plt.xscale('log')
plt.xlabel('λ (Regularization Parameter)')
plt.ylabel('Mean Squared Error')
plt.title('Bias-Variance Trade-off')
plt.legend()
plt.grid(True)
plt.show()

lambdas = [0, 0.01, 1, 100]
plt.scatter(X,y,color = 'blue',label ='Training Data',alpha=0.5)
for i,l in enumerate(lambdas):
    model = Ridge(alpha=l)
    model.fit(X_train, y_train)
    y_plot = model.predict(X_plot_poly)

    plt.plot(X_plot, y_plot, label=f'λ = {l}')

plt.title('Polynomial Regression for Different λ')
```

```
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```

```
Learned Parameters (θ): [ 0.          1.2882143  -0.56101382 -0.2448971   0.08034125 -
0.00525301]
```



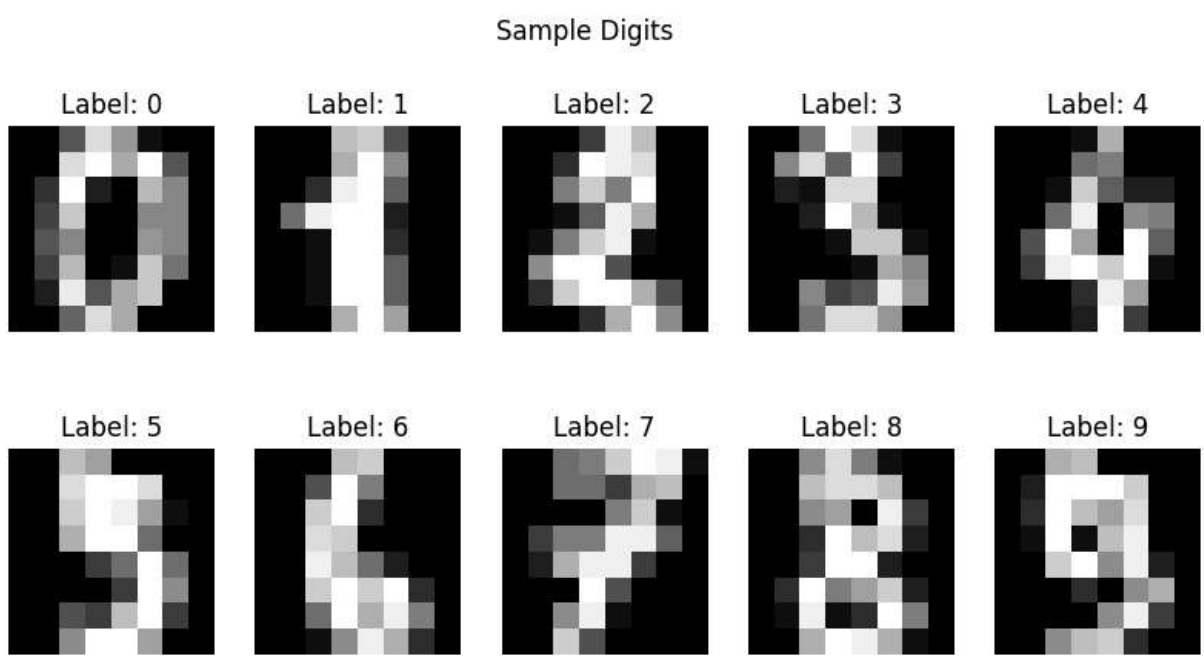

Bias-Variance Trade-off
Training MSE
Validation MSE
Mean Squared Error
λ (Regularization Parameter)
5.0
4.8
4.6
4.4
4.2
4.0
3.8
10^{-2}
10^{-1}
10^{0}
10^{1}
10^{2}


Polynomial Regression for Different λ
Training Data
λ = 0
λ = 0.01
λ = 1
λ = 100
y
x
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
0
2
4
6
8
10

Question 06

In [12]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load digits dataset
digits = load_digits()
X = digits.data  # shape: (1797, 64) — 64 features (8x8 images)
y = digits.target  # labels: 0-9

# Visualize some samples
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='gray',alpha = 1)
    ax.set_title(f'Label: {digits.target[i]}')
    ax.axis('off')
plt.suptitle('Sample Digits')
plt.show()

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Cost and gradient for logistic regression
def cost_function(theta, X, y, reg_lambda):
    m = len(y)
    h = sigmoid(X @ theta)
    epsilon = 1e-5  # for numerical stability
    cost = (-y @ np.log(h + epsilon) - (1 - y) @ np.log(1 - h + epsilon)) / m
    reg = (reg_lambda / (2 * m)) * np.sum(theta[1:] ** 2)
    return cost + reg

def gradient(theta, X, y, reg_lambda):
    m = len(y)
    h = sigmoid(X @ theta)
    grad = (X.T @ (h - y)) / m
    grad[1:] += (reg_lambda / m) * theta[1:]
    return grad

# Add intercept term
X = np.insert(X, 0, 1, axis=1)  # shape: (1797, 65)

# One-vs-All Training
def one_vs_all(X, y, num_labels, reg_lambda=0.1):
    m, n = X.shape
    all_theta = np.zeros((num_labels, n))

    from scipy.optimize import minimize
    for c in range(num_labels):
        y_c = (y == c).astype(int)  # binary for class c
        theta0 = np.zeros(n)
        res = minimize(fun=cost_function, x0=theta0, args=(X, y_c, reg_lambda),
                       jac=gradient, method='TNC', options={'maxiter': 300})
        all_theta[c] = res.x
```

```python
    return all_theta

# Prediction
def predict_one_vs_all(all_theta, X):
    probs = sigmoid(X @ all_theta.T)  # shape: (m, num_labels)
    return np.argmax(probs, axis=1)

# Train the OvA logistic regression
num_labels = 10  # digits 0-9
reg_lambda = 0.1
all_theta = one_vs_all(X, y, num_labels, reg_lambda)

# Predict and compute accuracy
y_pred = predict_one_vs_all(all_theta, X)
accuracy = accuracy_score(y, y_pred)

print("Training Accuracy (OvA):", accuracy * 100, "%")

# Display learned theta for each class (first 10 coefficients)
for i in range(num_labels):
    print(f"Class {i} θ (first 10):", all_theta[i][:10])

# Built-in Logistic Regression for comparison
clf = LogisticRegression(multi_class='ovr', solver='lbfgs', max_iter=300)
clf.fit(digits.data, digits.target)
sk_accuracy = clf.score(digits.data, digits.target)
print("Scikit-learn OvA Accuracy:", sk_accuracy * 100, "%")
```




```
C:\Users\Sabbir Ahmed\AppData\Local\Temp\ipykernel_21480\3684129091.py:53: OptimizeWar
ning: Unknown solver options: maxiter
  res = minimize(fun=cost_function, x0=theta0, args=(X, y_c, reg_lambda),
```

```
Training Accuracy (OvA): 99.8330550918197 %
Class 0 θ (first 10): [-1.50543180e+01  0.00000000e+00 -5.09630737e-02  2.41780761e-02
  1.02277029e-01 -2.70790968e-02 -4.63455382e-01 -3.79370759e-01
 -3.76596566e-02  9.86755012e-05]
Class 1 θ (first 10): [-6.66538597e+01  0.00000000e+00  6.99770774e-01  3.73908019e-01
  7.63325557e-01 -2.97162860e+00  2.87652762e+00 -1.94279193e+00
  3.03445926e-05  1.71748792e-07]
Class 2 θ (first 10): [-1.91372939e+01  0.00000000e+00  6.60166821e-02 -1.31365497e-01
  1.94370917e-01 -7.20183018e-02 -2.41299991e-01 -1.24738289e-01
 -1.28255309e-02 -5.09963125e-05]
Class 3 θ (first 10): [-1.94880378e+01  0.00000000e+00  3.29077947e-01 -1.77146512e+00
  3.96871374e-01  1.28409583e+00 -4.03440213e-01 -9.33562053e-01
  6.56415366e-01 -6.98052189e-03]
Class 4 θ (first 10): [ 5.84019073e+00  0.00000000e+00 -8.04374128e-03 -4.91206522e-01
 -1.05498178e+00 -4.76854571e-01 -4.52102710e-01 -6.50244717e-01
  2.04544828e-01 -4.81212225e-04]
Class 5 θ (first 10): [-2.55572318e+01  0.00000000e+00  1.16540466e-01  1.21325736e+00
 -4.99051396e-01  4.25573058e-02  7.83467448e-01  1.05519710e+00
 -3.66256749e-01  1.75760540e-03]
Class 6 θ (first 10): [-3.02321199e+01  0.00000000e+00 -8.42283471e-02 -2.07161034e-01
 -5.06198311e-01  1.71204141e-02 -4.27454248e-01 -2.81067320e-01
  1.27488172e-04 -6.39240398e-04]
Class 7 θ (first 10): [-8.86248612e+00  0.00000000e+00 -2.12396225e-01  5.82363170e-01
  4.12196959e-01 -2.32550983e-01  7.28047847e-01  4.62721362e-01
  3.32873974e-01 -2.92142249e-07]
Class 8 θ (first 10): [-1.01589267e+01  0.00000000e+00  1.07228730e-02  2.55324488e-01
 -2.51126793e-01  8.13986414e-02  4.29396894e-03 -6.50839795e-01
 -1.13859428e-01  2.21990388e+00]
Class 9 θ (first 10): [-4.00219900e+01  0.00000000e+00 -1.71055924e+00 -1.61072112e-02
  9.10883052e-01  9.02042742e-02 -3.28973762e-01  1.05092288e-01
 -1.41836176e-01 -2.98131823e-03]
```